

Code:

```
#include <stdio.h> #include <stdlib.h>

typedef struct QNode { struct QNode *prev, *next; unsigned pageNumber; } QNode;

typedef struct Queue { unsigned count; unsigned numberOfFrames; QNode *front,
*rear; } Queue;

typedef struct Hash { int capacity; QNode* *array; } Hash;

QNode* newQNode(unsigned pageNumber) { QNode* temp =
(QNode*)malloc(sizeof(QNode)); temp->pageNumber = pageNumber; temp->prev =
temp->next = NULL; return temp; }

Queue* createQueue(int numberOfFrames) { Queue* queue =
(Queue*)malloc(sizeof(Queue)); queue->count = 0; queue->front = queue->rear = NULL;
queue->numberOfFrames = numberOfFrames; return queue; }

Hash* createHash(int capacity) { Hash* hash = (Hash*)malloc(sizeof(Hash)); hash-
>capacity = capacity; hash->array = (QNode**)malloc(hash->capacity *
sizeof(QNode*)); for (int i = 0; i < hash->capacity; ++i) hash->array[i] = NULL; return
hash; }

int areAllFramesFull(Queue* queue) { return queue->count == queue-
>numberOfFrames; }

int isEmptyQueue(Queue* queue) { return queue->rear == NULL; }

void dequeueQueue(Queue* queue) { if (isEmptyQueue(queue)) return; if (queue->front
== queue->rear) queue->front = NULL; else queue->rear->prev->next = NULL;
free(queue->rear); queue->rear = queue->rear ? queue->rear->prev : NULL; queue-
>count--; }

void Enqueue(Queue* queue, Hash* hash, unsigned pageNumber) { if (hash-
>array[pageNumber] != NULL) { QNode* reqPage = hash->array[pageNumber]; if
(reqPage != queue->front) { if (reqPage == queue->rear) { queue->rear = reqPage->prev;
queue->rear->next = NULL; } else { reqPage->prev->next = reqPage->next; reqPage-
>next->prev = reqPage->prev; } reqPage->next = queue->front; reqPage->prev = NULL;
queue->front->prev = reqPage; queue->front = reqPage; } return; }

if (areAllFramesFull(queue)) {
    hash->array[queue->rear->pageNumber] = NULL;
    dequeueQueue(queue);
}
```

```

}

QNode* temp = newQNode(pageNumber);
temp->next = queue->front;
if (isEmpty(queue))
    queue->front = queue->rear = temp;
else {
    queue->front->prev = temp;
    queue->front = temp;
}
hash->array[pageNumber] = temp;
queue->count++;

}

int main() { int n, m; if (scanf("%d %d", &n, &m) != 2) return 0;

Queue* q = createQueue(m);
Hash* hash = createHash(100);

for (int i = 0; i < n; i++) {
    int page;
    scanf("%d", &page);
    Enqueue(q, hash, page);
}

QNode* curr = q->front;
while (curr != NULL) {
    printf("%d", curr->pageNumber);
    if (curr->next != NULL) printf(" ");
    curr = curr->next;
}
printf("\n");

return 0;

}

```

Input:

```

6 4
1 2 3 1 4 5

```

Output:

5 4 1 3